Lesson 7 - Filter Design

Now we have the job of actually making filters in MATLAB. Turns out that trying to make filters is pretty complicated, but MATLAB bundles it most of it up into the Signal Processing Toolbox for our use.

Designing Filters

Before making our filters, we have some design considerations to take into account with various tradeoffs to consider. For the purposes of this lesson, we will consider the following types of IIR filters.

Filter	Passband	Stopband
Butterworth	Monotonic	Monotonic
Chebyshev I	Equiripple	Monotonic
Chebyshev II	Monotonic	Equiripple
Elliptic	Equiripple	Equiripple

Table 1: Types of IIR Filters

When we have picked a type of filter, we also need the following specifications to actually make the filter:

- Filter order
- Passband attenuation ripple
- Stopband attenuation
- Passband/stopband edge frequencies

The edge frequencies for the passband and stopband also determine whether the filter is a highpass, lowpass, bandpass, or bandstop filter.

Making Filters

Let's say that we are designing a filter for audio applications and I want a filter that passes through the frequencies common in speech. We might use a Butterworth lowpass filter to make sure the passband and stopband don't distort the signal too much.

For the rest of the lesson, we will be designing a Butterworth filter. The process for the other IIR filters is very similar (but still read the documentation).

When designing filters in MATLAB, we typically give all the properties except filter order first and let MATLAB handle the rest.

```
fs = 44100; % Sampling rate of 44.1 kHz
fn = fs/2; % Nyquist frequency

```

```
% Lowpass filter
Wp = 3400 / fn;
Ws = 4500 / fn;

% Note: Frequencies have to be in terms of fractions of the Nyquist frequency.

Rp = 3; % Ripple in passband (dB)
Rs = 60; % Attenutation in stopband (dB)

[n, Wn] = buttord(Wp, Ws, Rp, Rs);
```

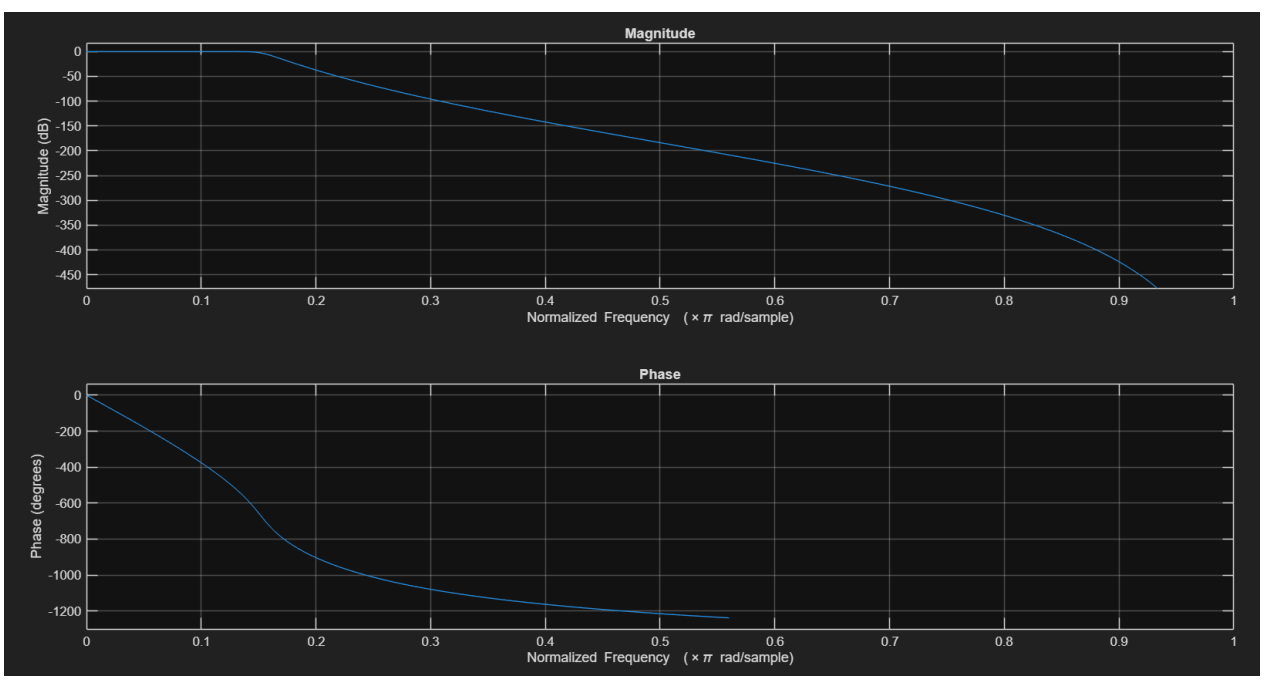
Once we have the order of the filter, we can plug the outputs into the corresponding function to make the filter. This inputs required from this step will vary for each type of filter, so do read the documentation for this.

```
[z, p, k] = butter(n, Wn, 'low');
% Note: Bandpass and bandstop filters that are made with these functions actually have order 2n.
```

We can then check the filter we just made with freqz() from last lesson.

Note: Professor Fontaine specifically does not want you to use the plot automatically generated by freqz() when doing the homework for signals. Plot them manually in that case.

```
[b, a] = zp2tf(z, p, k);
figure;
freqz(b, a, 1e3);
```



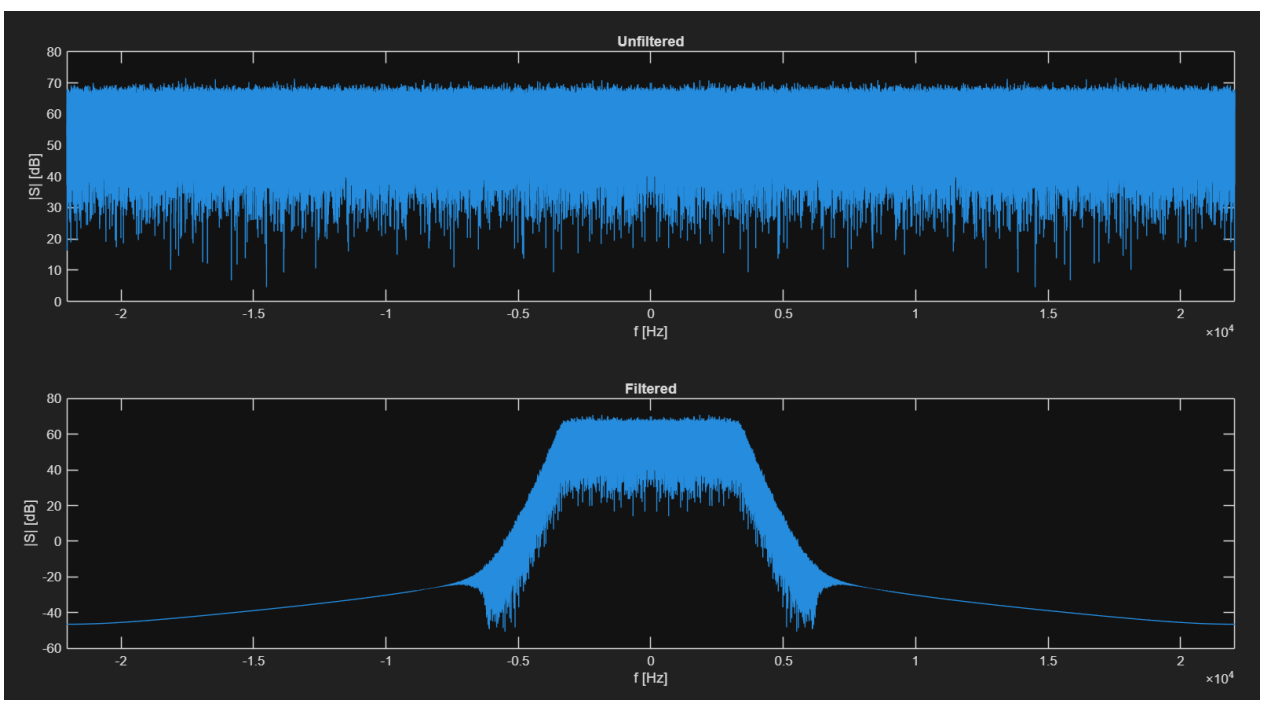
Using Filters

Now that we have a filter to use, we should use it to filter some signals. The first option in MATLAB is to filter the signal using the polynomial representation of the filter.

```
l = 1e6;
s = randn(1, l); % Making white noise to filter
S = fftshift(fft(s)); % Find fft
s_filt = filter(b, a, s); % Apply filter
S_filt = fftshift(fft(s_filt));

figure;
subplot(2, 1, 1);
plot(fs / l * (-l/2 : l/2-1), 20 * log10(abs(S)));
title('Unfiltered');
ylabel('|S| [dB]');
xlim([-1, 1] * fn);
xlabel('f [Hz]');

subplot(2, 1, 2);
plot(fs / l * (-l/2 : l/2- 1), 20 * log10(abs(S_filt)));
title('Filtered');
ylabel('|S| [dB]');
xlim([-1, 1] * fn);
xlabel('f [Hz]');
```



Second-Order-Section Representations

When numerical stability becomes an issue, we can always use second-order-sections to represent our filters. Instead of representing the entire numerator and denominator at once, we can break the transfer function into factors where the numerator and denominator are at most degree 2, which is typically expressed as

$$H(z) = k(H_1(z)...H_n(z))$$

where

$$H_k(z) = \frac{b_{k0} + b_{k1}z^{-1} + b_{k2}z^{-2}}{1 + a_{k1}z^{-1} + a_{k2}z^{-2}}.$$

This is typically represented in MATLAB as a matrix with 6 columns where each row represents the coefficients of one of the factors.

$$\begin{pmatrix} b_{10} & b_{11} & b_{12} & 1 & a_{11} & a_{12} \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ b_{n0} & b_{n1} & b_{n2} & 1 & a_{n1} & a_{n2} \end{pmatrix}$$

We can use filter our signal using second-order-sections in MATLAB like this.

```
[sos, g] = zp2sos(z, p, k);
s_filt = g * sosfilt(sos, s);
```

Further Reading

If you want to learn more about filter design/analysis in MATLAB, I would recommend taking a look at `https://www.mathworks.com/help/signal/filter-design.html` (or take Digital Signal Processing).